E5FR
Année 2025 - 2026

Rapport de projet
Gestion des applications
Cloud-Native avec Kubernetes

Groupe: Yasin GUNDOGDU (yasin.gundogdu@edu.esiee.fr)
& Mike CUNHA (mike.cunha@edu.esiee.fr)
Lien du dépôt GitHub : https://github.com/Warrikard/Projet-Cloud-Security

Sommaire

Sommaire **2**
Introduction **3**
Phase 1 : Architecture et Déploiement **4**
1.1. L'architecture de l'application 4
1.2. Déploiement sur le cluster Kubernetes (K3s) 5
1.3. Gestion des Configurations et des Secrets 8
Phase 2 : Automatisation de l'intégration continue (CI/CD) **9**
Phase 3 : Gestion des Ressources et observabilité **13**
3.1. Quotas et Limites de Ressources 13
3.2. Monitoring (Prometheus & Grafana) 13
Phase 4 : Renforcement de la Sécurité **15**
4.1. Authentification RBAC Granulaire 15
4.2. Isolation Réseau (Network Policies) 15
4.3. Scans de Vulnérabilité (Trivy) 15
4.4. Sauvegardes Automatisées (Backups) 18
Conclusion **19**

Introduction

Dans le cadre de notre formation, ce projet a pour objectif de concevoir, déployer et administrer une architecture logicielle moderne en appliquant rigoureusement les méthodologies **DevOps** et les principes du **Cloud-Native**.

L'application, développée et déployée au cours de ce projet s'intitule “**Le Cloud Gourmand”**. Il s'agit d'un menu de restaurant interactif reposant sur une architecture distribuée à trois niveaux (3-tiers) :

- **Un Frontend** (Interface Utilisateur) servi par un serveur web léger Nginx.
- **Un Backend** (API) développé en Node.js permettant de faire le pont entre l'interface et les données.
- **Une Base de données** relationnelle PostgreSQL pour le stockage persistant des plats du menu.

Afin de garantir la scalabilité, la haute disponibilité et la portabilité de cette application, l'ensemble des services a été conteneurisé à l'aide de **Docker**, puis orchestré localement sur un cluster **Kubernetes** (via la distribution légère K3s).

Ce rapport technique retrace et documente pas à pas la réalisation de ce projet à travers quatre phases majeures de l'ingénierie Cloud :

1. **L'architecture et le déploiement** des composants Kubernetes de base (Pods, Services, ConfigMaps, Secrets).
2. **L'automatisation (CI/CD)** du cycle de vie de l'application via des pipelines GitHub Actions.
3. **L'observabilité et l'optimisation des ressources** grâce à l'intégration de la stack de monitoring Prometheus et Grafana (analyse des modèles RED et USE).
4. **Le renforcement de la sécurité** globale du cluster (politiques RBAC, isolation réseau, scans de vulnérabilités avec Trivy et stratégies de sauvegarde automatisées).

L'objectif final de ce document est de démontrer techniquement la viabilité, la résilience et le niveau de sécurité d'une infrastructure conteneurisée prête pour un environnement de production.

Phase 1 : Architecture et Déploiement

1.1. L'architecture de l'application

Notre application, "Le Cloud Gourmand", est divisée en trois microservices distincts pour garantir la scalabilité :

- **Frontend :** Un serveur web Nginx (port 80) servant l'interface utilisateur.
- **Backend :** Une API Node.js (port 3000) traitant les requêtes.
- **Base de données :** Un serveur PostgreSQL stockant le menu.

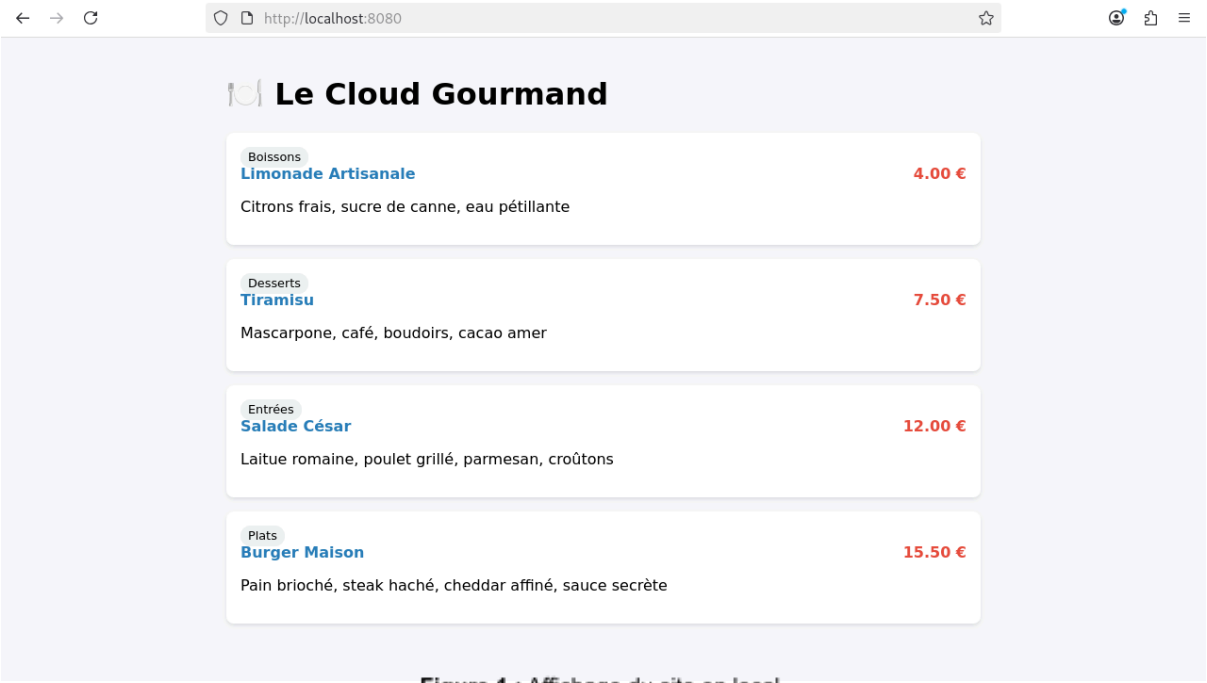

Figure 1 : Affichage du site en local

1.2. Déploiement sur le cluster Kubernetes (K3s)

Nous avons utilisé la distribution légère K3s pour héberger notre infrastructure. Tous les services ont été conteneurisés et déployés sous forme de Pods et exposés via des Services de type LoadBalancer ou ClusterIP.

```
vboxuser@Debian-GIT:~/Projet-Cloud-Security/Cloud_Native_Devops_With_Kubernetes/kubernetes_projet/fichiers$ kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
backend-deployment-6885677985-vm8nh   1/1     Running   0          32s
frontend-deployment-bb5dc9d6-mdzvm    1/1     Running   0          33s
postgres-statefulset-0                1/1     Running   0          33s
```

Figure 2 : Capture d'écran de la liste des pods de l'application

L'installation s'est effectuée via le script officiel, nous donnant accès à la commande k3s et configurant notre nœud local.

```
NAME:
   k3s - Kubernetes, but small and simple

USAGE:
   k3s [global options] command [command options]

VERSION:
   v1.34.4+k3s1 (c6017918)

COMMANDS:
   server           Run management server
   agent            Run node agent
   kubectl          Run kubectl
   crictl           Run crictl
   ctr              Run ctr
   check-config     Run config check
   token            Manage tokens
   etcd-snapshot    Manage etcd snapshots
   secrets-encrypt  Control secrets encryption and keys rotation
   certificate      Manage K3s certificates
   completion       Install shell completion script
   help, h          Shows a list of commands or help for one command
```

Figure 3 : Infos et commandes utilisable avec k3s

Notre machine contient alors un seul nœud (control-plane) prêt à accueillir nos charges de travail :

```
vboxuser@Debian-GIT:~/Projet-Cloud-Security$ kubectl get node
NAME          STATUS   ROLES           AGE   VERSION
debian-git    Ready    control-plane   21h   v1.34.5+k3s1
```

Figure 4 : Vérification de l'état du cluster Kubernetes affichant le nœud maître "control-plane"

Le déploiement complet de l'infrastructure s'est fait de manière déclarative en appliquant l'ensemble de nos fichiers de configuration en une seule fois :

```
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl apply -f namespace.yaml
namespace/cloud-native-project created
resourcequota/project-quota created
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl apply -f .
deployment.apps/backend-deployment created
service/backend-service created
cronjob.batch/postgres-backup created
deployment.apps/postgres-deployment created
service/postgres-service created
deployment.apps/frontend-deployment created
service/frontend-service created
configmap/db-init-script created
namespace/cloud-native-project unchanged
resourcequota/project-quota unchanged
networkpolicy.networking.k8s.io/db-security-policy created
secret/db-credentials created
configmap/app-config created
serviceaccount/backend-sa created
role.rbac.authorization.k8s.io/backend-config-reader created
rolebinding.rbac.authorization.k8s.io/backend-rolebinding created
```

Figure 5 : Capture d'écran des commandes de déploiement de l'application

Voici un aperçu de nos manifestes de déploiement pour le frontend et le backend :

Concernant le backend, c'est un deployment également, avec 1 réplique. Il écoute au port 80, et il nécessite au moins 128 mo de ram avec une limite à 256mo. Puisque c'est un serveur backend nodejs, il doit écouter au port 3000.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
  namespace: cloud-native-project
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: yassagan/monfrontend:latest
        resources:
          requests:
            memory: "64Mi"
            cpu: "50m"
          limits:
            memory: "128Mi"
            cpu: "100m"
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: cloud-native-project
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: cloud-native-project
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      serviceAccountName: backend-sa
      containers:
      - name: backend
        image: yassagan/monbackend:latest
        resources:
          requests:
            memory: "128Mi"
            cpu: "100m"
          limits:
            memory: "256Mi"
            cpu: "250m"
        ports:
        - containerPort: 3000
        envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: db-credentials
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: cloud-native-project
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
```

Figure 6 : Affichage des fichier de configuration Frontend/Backend

1.3. Gestion des Configurations et des Secrets

Afin de respecter les bonnes pratiques Cloud-Native, nous avons externalisé la configuration. Les variables d'environnement non sensibles (comme les ports et hôtes) sont gérées via un objet ConfigMap (app-config), tandis que les identifiants de la base de données sont sécurisés via un objet Secret (db-credentials).

```
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl get configmaps,secrets -n cloud-native-project
NAME                          DATA   AGE
configmap/app-config          3      92m
configmap/db-init-script      1      92m
configmap/kube-root-ca.crt    1      92m

NAME                     TYPE     DATA   AGE
secret/db-credentials    Opaque   3      92m
```

Figure 7 : Visualisation des ConfigMaps et des Secrets dans cloud-native-project

Dans le fichier de configuration de notre backend (backend.yaml), l'application fait appel à la ConfigMap app-config :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: cloud-native-project
data:
  DB_HOST: "postgres-service"
  DB_PORT: "5432"
---
```

Figure 8 : Visualisation de la configuration de ConfigMaps dans RBAC.yaml

Ces variables d'environnement sont ensuite dynamiquement récupérées dans le code source de l'Application Node.js :

```
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  user: process.env.POSTGRES_USER || 'admin',
```

Figure 9 : Visualisation de la configuration des variable récupérée

Phase 2 : Automatisation de l'intégration continue (CI/CD)

Avant d'automatiser le processus, nous avons validé la création (build) et l'envoi (push) manuel des images Docker vers notre espace de stockage en ligne.

```
vboxuser@Debian-GIT:~/projet-cloud$ docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: VHLB-NNDK
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser…
https://login.docker.com/activate

WARNING! Your credentials are stored unencrypted in '/home/vboxuser/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

Figure 10 : Validation de l'authentification Docker sur l'environnement de développement

Notre espace Docker Hub a ainsi été initialisé avec succès pour accueillir les images de notre frontend et de notre backend :

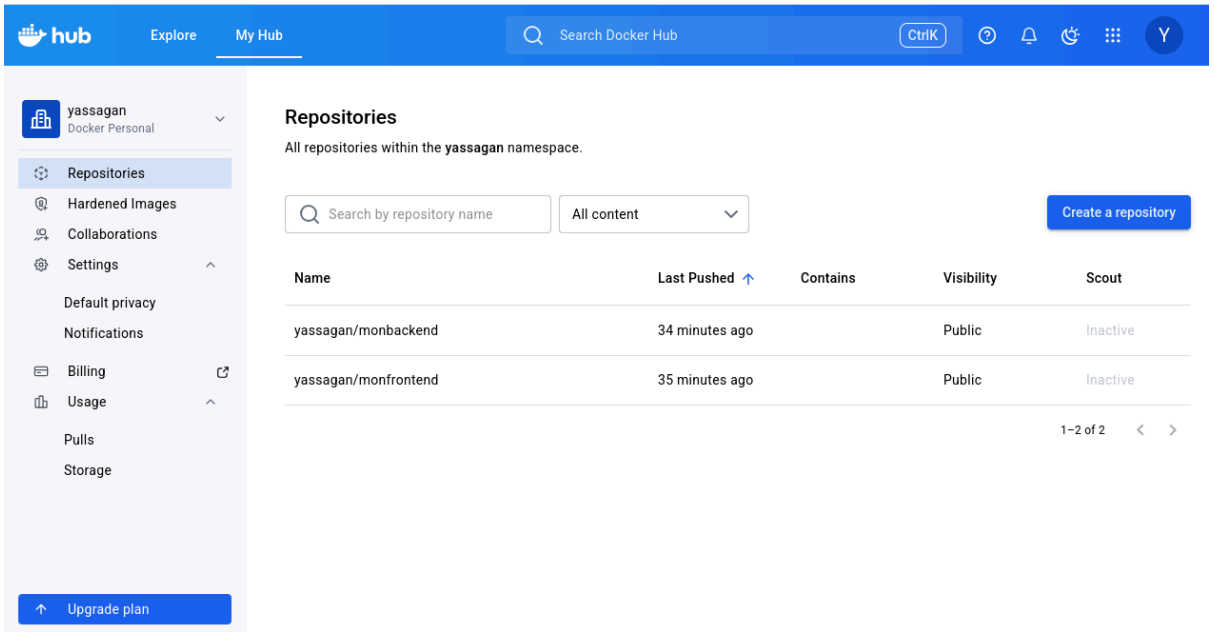

Figure 11 : État des dépôts sur Docker Hub après le déploiement des images monbackend et monfrontend

Pour respecter la méthodologie DevOps, nous avons mis en place un pipeline d'Intégration Continue (CI) automatisé à l'aide de GitHub Actions.

À chaque modification poussée (push) sur la branche principale du dépôt, le pipeline récupère le code source, construit les nouvelles images Docker pour le frontend et le backend, puis les publie automatiquement et de manière sécurisée sur Docker Hub.

Ouvrir

ci.yaml
~/Projet-Cloud-Security/.github/workflows

```yaml
# .github/workflows/ci.yml
name: Pipeline CI - Build & Push Docker

on:
  push:
    branches: [ "main", "master" ] # Se déclenche quand on envoie du code sur la branche principale

jobs:
  build-et-push:
    name: Création des images Docker
    runs-on: ubuntu-latest

    steps:
      # 1. On récupère le code source du projet
      - name: Checkout du code
        uses: actions/checkout@v3

      # 2. On se connecte au compte Docker Hub
      - name: Connexion à Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      # 3. On construit et on envoie le Backend
      - name: Build & Push Backend
        uses: docker/build-push-action@v4
        with:
          context: ./Cloud_Native_Devops_With_Kubernetes/kubernetes_projet/app/backend
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/monbackend:latest

      # 4. On construit et on envoie le Frontend
      - name: Build & Push Frontend
        uses: docker/build-push-action@v4
        with:
          context: ./Cloud_Native_Devops_With_Kubernetes/kubernetes_projet/app/frontend
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/monfrontend:latest
```

Figure 12 : Configuration du pipeline CI via un workflow GitHub Actions (ci.yml)

Le bon fonctionnement de ce workflow permet de s'affranchir des compilations manuelles.

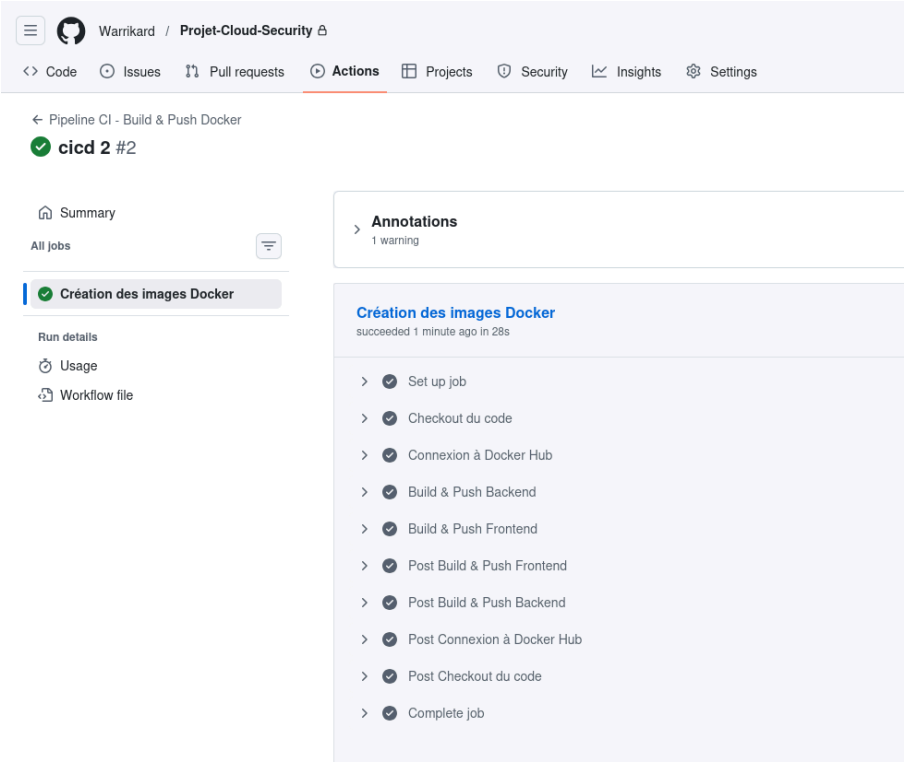

Figure 13 : Validation visuelle du build et du push des images Docker

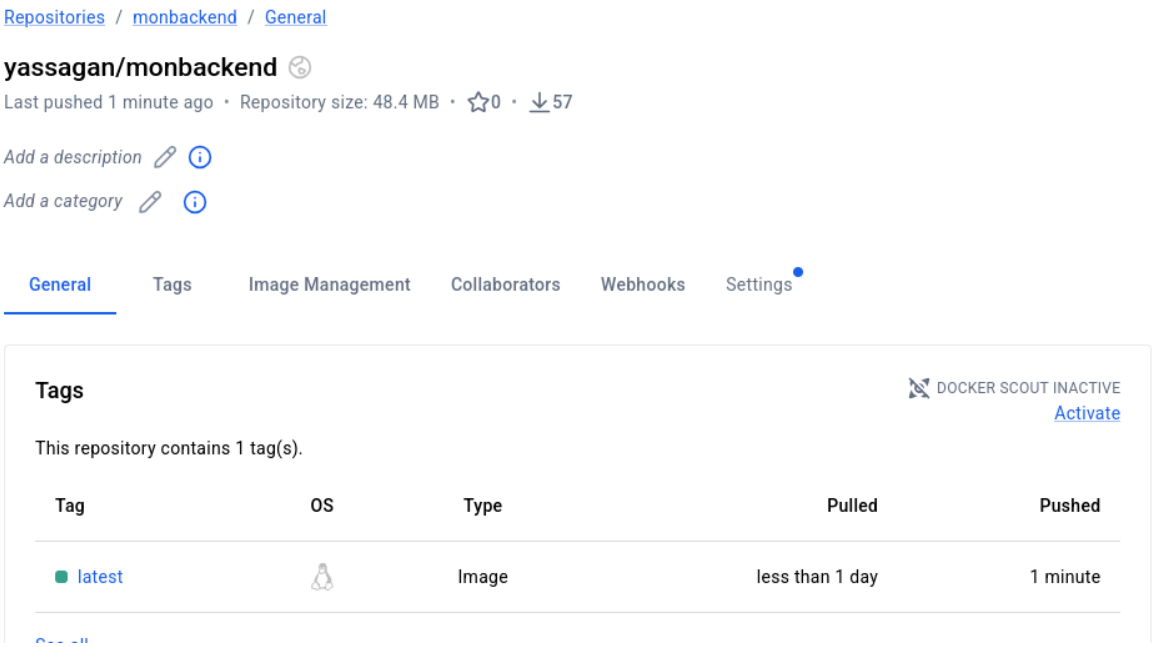

Figure 14 : Détails du dépôt monbackend affichant le tag latest récemment mis à jour

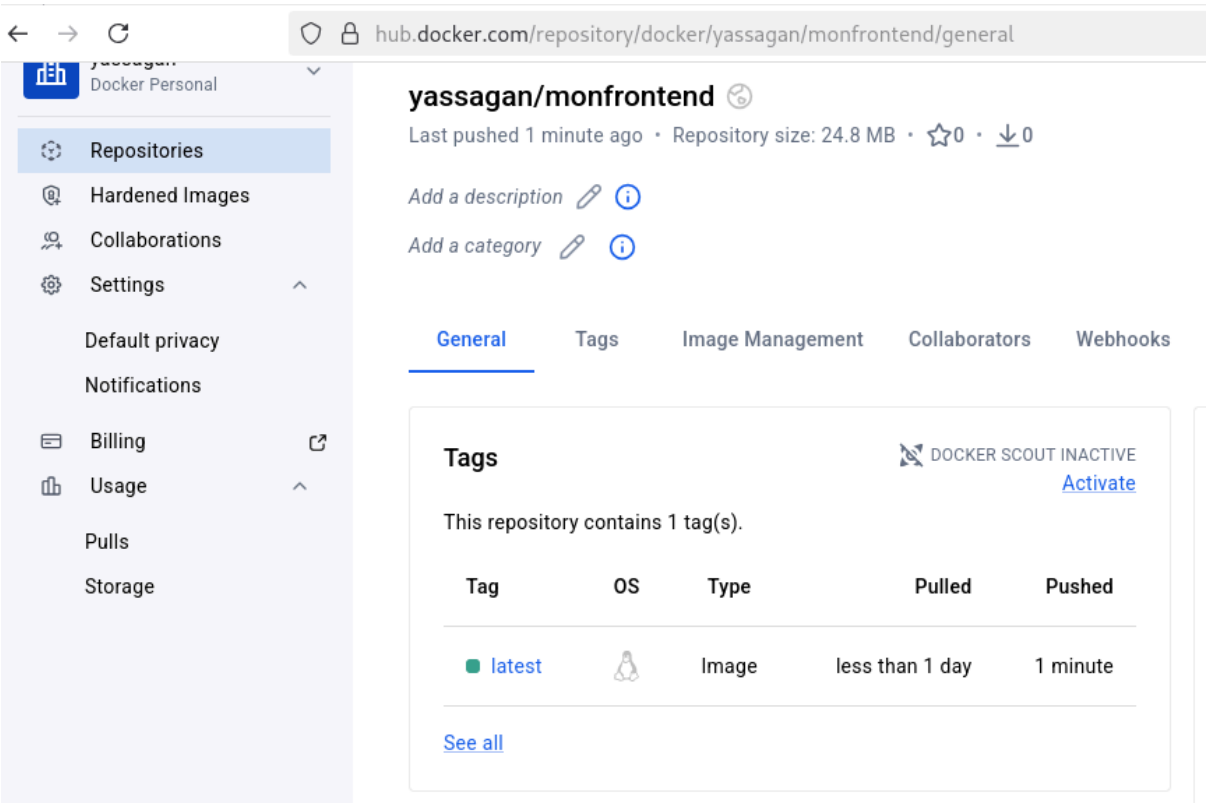

Figure 15 : Détails du dépôt monfrontend affichant le tag latest récemment mis à jour

Phase 3 : Gestion des Ressources et observabilité

3.1. Quotas et Limites de Ressources

Afin d'éviter qu'un conteneur défaillant ne monopolise toutes les ressources du cluster K3s, nous avons défini un ResourceQuota au niveau de l'espace de noms (Namespace) cloud-native-project, limitant l'utilisation globale du CPU et de la mémoire. Notre quota définit des limites strictes appliquées individuellement à chaque Pod, plafonnant la consommation totale à **2 cœurs CPU** et **2 Gi de RAM** pour l'ensemble des services. Il garantit une réservation minimale de **1 CPU** et **1 Gi de mémoire**, tout en limitant le nombre maximal de conteneurs à **15 pods**. Cette configuration assure une cohabitation saine des ressources en empêchant tout débordement technique qui pourrait saturer le nœud Debian.

```
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl describe quota -n cloud-native-project
Name:            project-quota
Namespace:       cloud-native-project
Resource         Used   Hard
--------         ----   ----
limits.cpu       600m   2
limits.memory    640Mi  2Gi
pods             3      15
requests.cpu     250m   1
requests.memory  320Mi  1Gi
```

Figure 16 : État de consommation du project-quota montrant les ressources utilisées

3.2. Monitoring (Prometheus & Grafana)

Pour la supervision en temps réel, la stack Kube-Prometheus a été déployée. Elle permet d'analyser les performances selon les standards de l'industrie :

Le modèle USE (Utilization, Saturation, Errors) : Permet le suivi de l'utilisation CPU et RAM de nos conteneurs.

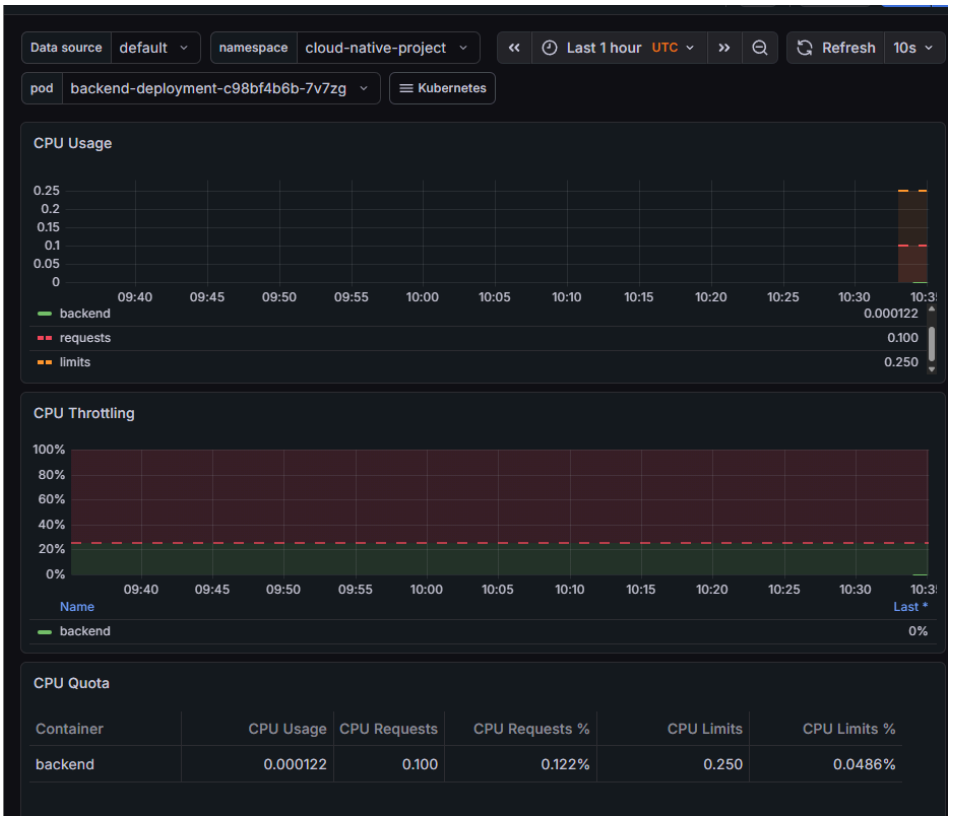

Figure 17 : Suivi des performances du Pod backend

Le modèle RED (Rate, Errors, Duration) : Permet le suivi du trafic réseau et de la bande passante de l'application.

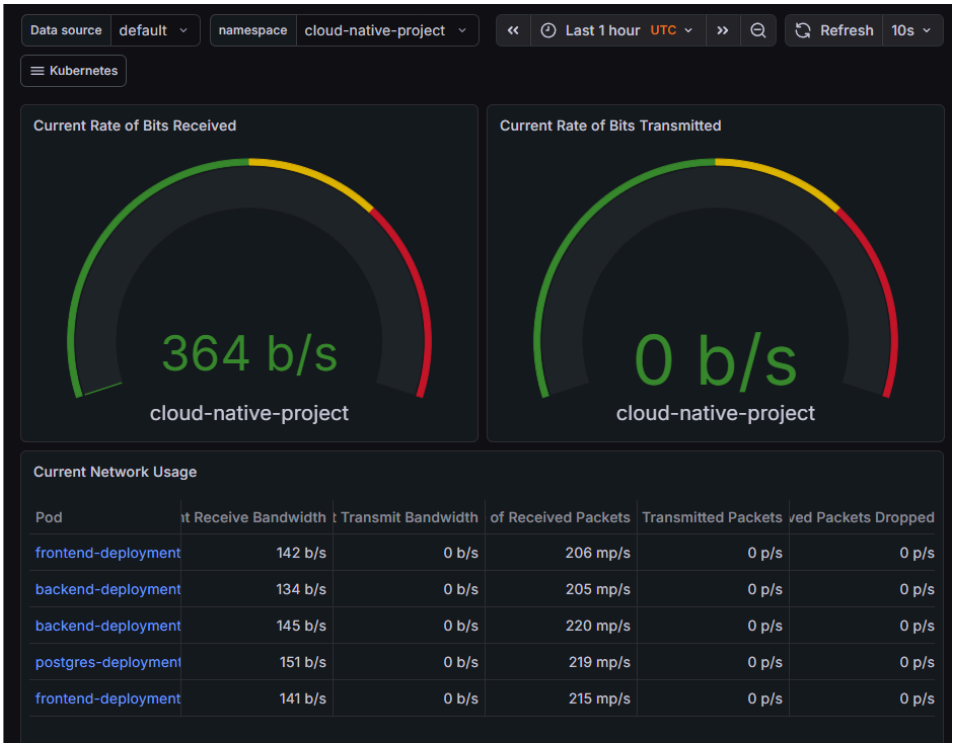

Figure 18 : Suivi Analyse des flux réseau entrants et sortants par service via le dashboard Grafana

Phase 4 : Renforcement de la Sécurité

4.1. Authentification RBAC Granulaire

Nous avons appliqué le principe de moindre privilège. Un ServiceAccount spécifique a été créé pour le backend, couplé à un RoleBinding qui lui donne uniquement les droits stricts dont il a besoin, l'empêchant de modifier l'état du cluster.

```
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl get roles,rolebindings,serviceaccounts -n cloud-native-project
NAME                                                  CREATED AT
role.rbac.authorization.k8s.io/backend-config-reader   2026-03-27T17:27:45Z

NAME                                                        ROLE                          AGE
rolebinding.rbac.authorization.k8s.io/backend-rolebinding   Role/backend-config-reader    98m

NAME                         SECRETS   AGE
serviceaccount/backend-sa    0         98m
serviceaccount/default       0         98m
```

Figure 19 : Suivi Vérification des ressources RBAC

4.2. Isolation Réseau (Network Policies)

Pour protéger la base de données de toute intrusion, une NetworkPolicy nommée db-security-policy a été mise en place. Elle bloque tout le trafic entrant vers PostgreSQL, à l'exception stricte des requêtes provenant du pod Backend (port 5432).

```
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ cat networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-security-policy
  namespace: cloud-native-project
spec:
  podSelector:
    matchLabels:
      app: postgres
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 5432
```

Figure 20 : Définition d'une NetworkPolicy pour restreindre l'accès à la base de données PostgreSQL

4.3. Scans de Vulnérabilité (Trivy)

L'outil Trivy a été utilisé pour auditer la sécurité de notre infrastructure. Il a permis de scanner nos images Docker et nos configurations Kubernetes afin d'identifier les failles de sécurité connues (CVE) et les corriger.

Scan de l’image frontend:

```
Total: 5 (UNKNOWN: 0, LOW: 0, MEDIUM: 3, HIGH: 1, CRITICAL: 1)

libexpat | CVE-2026-32767 | CRITICAL | fixed | 2.7.4-r0 | 2.7.5-r0
SiYuan: Authorization Bypass Allows Arbitrary SQL Execution
via Search API
https://avd.aquasec.com/nvd/cve-2026-32767
```

Figure 20 : Analyse de sécurité de l'image : identification d'une vulnérabilité critique

On peut observer qu’il y a une faille critique identifiée sous la référence CVE-2026-32767 dans la bibliothèque libexpat. Cette vulnérabilité permet un contournement d'autorisation et l'exécution de requêtes SQL arbitraires via l'API de recherche, ce qui nécessite une mise à jour immédiate vers la version 2.7.5-r0 pour sécuriser l'image.

Scan de l’image du backend :

```
Node.js (node-pkg)

Total: 13 (UNKNOWN: 0, LOW: 2, MEDIUM: 0, HIGH: 11, CRITICAL: 0)
```

Figure 20 : Rapport de scan Trivy révélant le nombre de vulnérabilité trouvé par niveau

On peut observer que l'image Node.js présente un total de 13 vulnérabilités. Bien qu'aucune ne soit classée comme critique, la présence de 11 failles de niveau "HIGH" (Élevé) indique une surface d'attaque importante qui nécessite une révision des dépendances du package node-pkg ou l'utilisation d'une image de base plus récente et sécurisée.

Scan des ressources du cluster :

Summary Report for default

Workload Assessment

Namespace	Resource	Vulnerabilities					Misconfigurations				
		C	H	M	L	U	C	H	M	L	U
cloud-native-project	Service/backend-service									1	
cloud-native-project	Service/frontend-service									1	
cloud-native-project	Service/postgres-service									1	
cloud-native-project	ConfigMap/app-config								1		
cloud-native-project	Deployment/backend-deployment	2	15	21	7			4	5	8	
cloud-native-project	Deployment/frontend-deployment	1	1	3				3	6	7	
cloud-native-project	Deployment/postgres-deployment	1	6	13	1			3	3	7	
cloud-native-project	NetworkPolicy/db-security-policy									1	

Severities: C=CRITICAL H=HIGH M=MEDIUM L=LOW U=UNKNOWN

Infra Assessment

Namespace	Resource	Vulnerabilities					Misconfigurations				
		C	H	M	L	U	C	H	M	L	U

Severities: C=CRITICAL H=HIGH M=MEDIUM L=LOW U=UNKNOWN

Figure 20 : État de sécurité des ressources Kubernetes par niveau de sévérité (Critique à Faible)

On peut également voir que l'ensemble du namespace cloud-native-project présente des risques de configuration. Les déploiements (backend, frontend et postgres) cumulent non seulement des vulnérabilités logicielles (colonnes C/H/M/L), notamment 4 critiques et plus de 20 élevées, mais aussi plusieurs erreurs de configuration (Misconfigurations). Ces défauts pourraient concerner des privilèges root excessifs ou l'absence de limites de ressources au sein du cluster.

4.4. Sauvegardes Automatisées (Backups)

La résilience des données est assurée par un objet CronJob Kubernetes. Celui-ci est configuré pour déclencher un conteneur éphémère chaque nuit à 2h00 du matin, qui exécute un dump (sauvegarde) complet de la base de données PostgreSQL.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: postgres-backup
  namespace: cloud-native-project
spec:
  schedule: "0 2 * * *" # Tous les jours à 2h00 du matin
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: pg-backup
            image: postgres:15-alpine
            command:
            - /bin/sh
            - -c
            - "pg_dump -h $DB_HOST -p $DB_PORT -U $POSTGRES_USER -d
$POSTGRES_DB -F c -f /tmp/db_backup.dump && echo 'Backup terminé avec
succès !'"
            envFrom:
            - configMapRef:
                name: app-config
            - secretRef:
                name: db-credentials
          restartPolicy: OnFailure
kubectl get cronjobs.batch
NAME              SCHEDULE    TIMEZONE   SUSPEND   ACTIVE   LAST
SCHEDULE   AGE
postgres-backup   0 2 * * *   <none>     False     0        <none>
29s
```

Figure 21 : Configuration et statut d'un CronJob de sauvegarde PostgreSQL

```
bash: ubectl : commande introuvable
vboxuser@Debian-GIT:~/projet-cloud/fichiers$ kubectl get cronjobs -n cloud-native-project
NAME              SCHEDULE    TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
postgres-backup   0 2 * * *   <none>     False     0        <none>          101m
```

Conclusion

Ce projet de déploiement de l'application "Le Cloud Gourmand" a permis de mettre en pratique les concepts fondamentaux des architectures Cloud-Native. En partant d'un code source basique, nous avons conçu une infrastructure complète, résiliente et orchestrée par Kubernetes (K3s).

Nous avons pu valider avec succès chaque pilier de la méthodologie DevOps : l'automatisation de nos déploiements via un pipeline CI/CD (GitHub Actions) , l'observabilité fine de nos conteneurs grâce à Prometheus et Grafana (analyse des modèles RED et USE), ainsi que le durcissement de notre sécurité (politiques RBAC, isolation réseau, audits Trivy et sauvegardes automatisées).

En définitive, ce travail nous a apporté une maîtrise technique globale des outils standards de l'industrie (Docker, Kubernetes, Helm) et nous a préparés aux enjeux réels de la gestion de microservices en production.